Zeyad Mohamed Elsonney
Electrical and Computer Engineering · October 11, 2026

Fixed-Point Moving-Average Filter

Non-recursive, recursive and counter-refreshed implementations: an SQNR study (M = 32)

1. Overview

This document describes a MATLAB study of a **moving-average (boxcar) filter** of length **M = 32** operating on 8-bit signed samples, and asks one hardware question: *how can the filter be built cheaply without letting its accuracy collapse?* Three architectures are compared:

1. **Non-recursive form**: sum the last M samples every cycle with an adder tree. Accurate, but expensive.
2. **Recursive form**: update the previous output with one add and one subtract. Cheap, but quantisation error accumulates without bound.
3. **Recursive form with a counter-controlled refresh**: run the cheap recursion, and every C samples reload the output with an exact full sum computed by a small **serial adder**.

1.1 Method and figure of merit

The model feeds 100,000 random integers drawn uniformly from −128…127 into each filter. Samples are normalised to x_n = x / 128 (a Q1.7 number) and the output is quantised to a chosen word length. Quality is measured as the signal-to-quantisation-noise ratio:

$$SQNR = 10 \cdot \log_{10}\left(P_y / P_e \right) \quad with \;\; P_y = mean(y_ideal^2),\;\; P_e = mean((y - y_ideal)^2)$$

where y_ideal is the unquantised non-recursive output (`y_non_rec` in the script). The design target is **SQNR ≥ 30 dB**. Time plots use sliding power windows (2000, 400 and 200 samples for the non-recursive, recursive and refresh plots respectively); the counter sweep uses the whole run.

Table 1. Simulation parameters.

Parameter	Value used in the MATLAB model
Window length M	32 (division by M is a 5-bit right shift)
Input	100,000 samples, uniform integers −128…127, normalised by 128
Output word length ("Bits")	8 to 13 bits, i.e. 7 to 12 fractional bits (`T_power = 7 … 12`, bits = T_power + 1)
Refresh counter sizes C	2, 4, 8, 16, 32, 64, 128, 256, 512 samples
SQNR target	30 dB
Rounding	MATLAB `round()` (ties rounded away from zero) after every operation

Summary of findings

- The **non-recursive** form reaches the 30 dB target already at 8 bits (32.24 dB), but needs 31 adders and a 31-register delay line.
- The **recursive** form needs one add and one subtract per sample, yet its SQNR **falls with time** and averages about **−32 dB** for every word length up to 12 bits: the accumulated error is larger than the signal.

- A **refresh every C samples** bounds the error. The 30 dB target is met at 8 bits with C = 2, 9 bits with C ≤ 8, 10 and 11 bits with C ≤ 32, and 12 bits with C ≤ 64.
- At **13 bits** (8 input bits + 5 growth bits) the output grid is fine enough to hold the exact average, so the rounding error is zero and the SQNR reads ≈120 dB for every architecture and every C (a numerical floor, see Section 8).
- The refresh sum is computed by **one serial adder** and a small FSM instead of a 31-adder tree, and the recursive path keeps running while it works, so no extra latency is added to the output.

2. Non-Recursive Form

2.1 Definition and block diagram

The direct (finite-impulse-response) form recomputes the full sum of the last M samples at every clock:

$$y[n] = (1 / M) \cdot \Sigma\, x[n - k] \qquad k = 0 \ldots M - 1, \quad M = 32$$

In the MATLAB script the window is written forward (`sum(x_n(i:i+M-1))/M`). That is the same filter delayed by M − 1 samples, so it has identical SQNR. Division by M = 32 is a 5-bit shift and costs no logic.

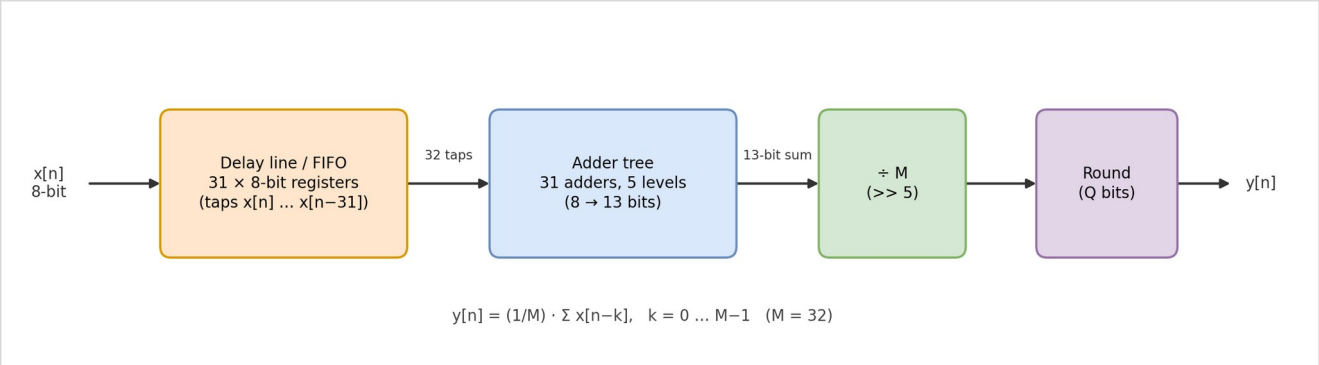


Figure 1. *Block diagram of the non-recursive moving average.*

2.2 Adder tree and delay line

All 32 taps are needed at the same instant, so the sum is built as a balanced binary adder tree: 16 + 8 + 4 + 2 + 1 = **31 adders** in **$\log_2 32 = 5$ levels**. The operand width grows by one bit per level (9, 10, 11, 12, 13 bits) because the sum of 32 eight-bit values needs 8 + 5 = 13 bits. The tree is drawn in full on the next page, together with the delay line that feeds it.

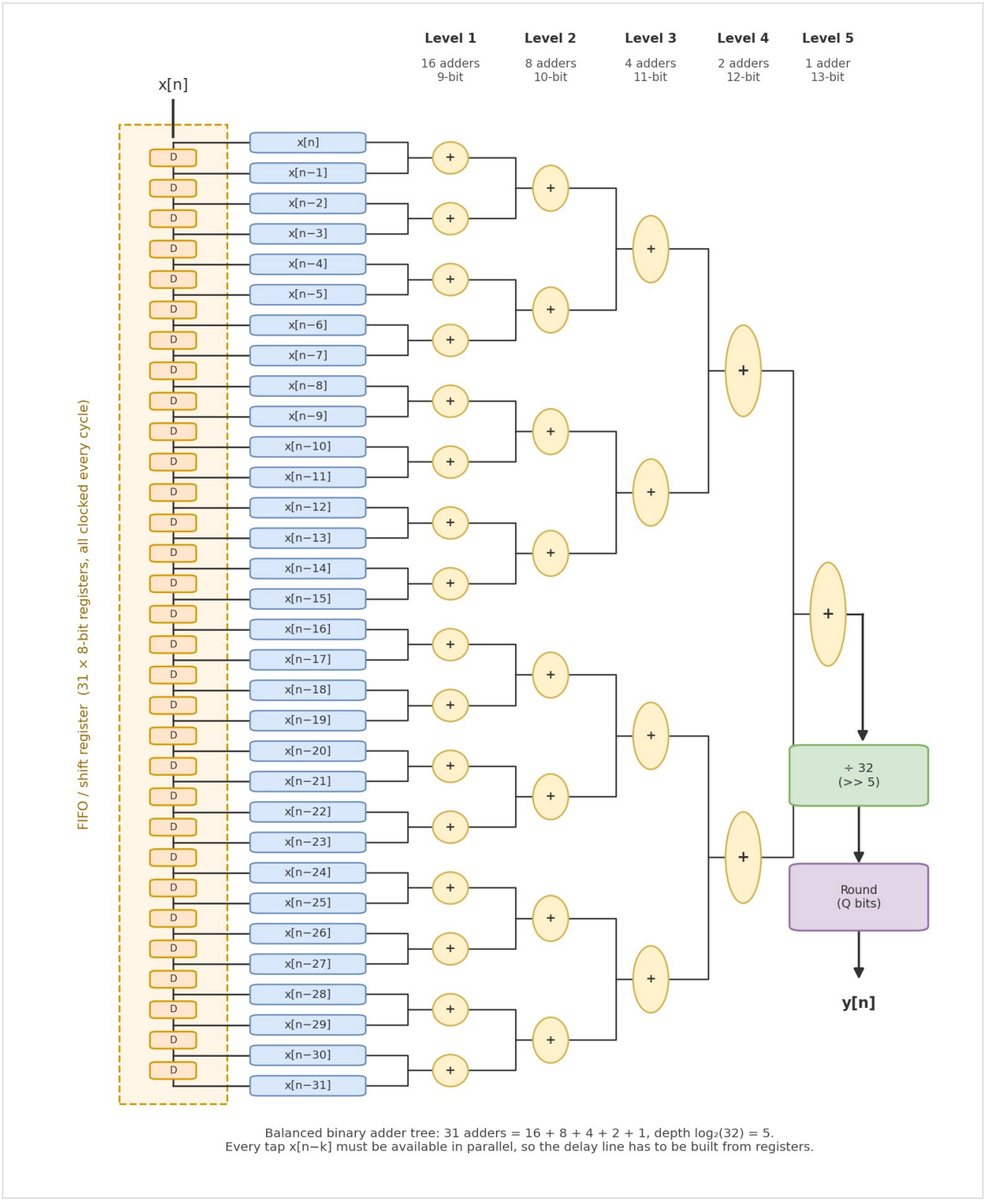


Figure 2. *Non-recursive moving average: 31-register delay line (FIFO) feeding a 5-level, 31-adder tree, followed by the ÷32 shift and the output rounding.*

2.3 Why the FIFO and the tree are expensive

The input history has to be stored somewhere. Calling it a FIFO or a shift register does not change the cost, because **every one of the 32 taps must be read in parallel**:

- **Storage:** 31 × 8 = 248 flip-flops (plus the input register). A RAM-based FIFO would need 32 simultaneous read ports, so the history must be built from registers.

- **Switching power:** in a shift-register FIFO every flip-flop captures new data on every clock, so dynamic power grows linearly with M.
- **Adders:** 31 adders with widths growing from 9 to 13 bits. As a ripple-carry estimate that is 16·9 + 8·10 + 4·11 + 2·12 + 13 = **305 full-adder cells**.
- **Timing:** the combinational path crosses five adders. Meeting a fast clock means pipelining the tree, which costs roughly another 305 flip-flops and **5 cycles of latency**.
- **Scaling:** doubling M doubles the delay line and adds M more adders. Cost grows linearly with the window length.

Table 2. Resource estimate for the non-recursive form.

Resource (M = 32)	Delay line / FIFO	Adder tree
Count	31 registers × 8 bits = 248 flip-flops	31 adders in 5 levels
Access pattern	All 32 taps read every cycle	All 31 adders active every cycle
Width	8 bits	9 → 13 bits (one more bit per level)
Timing / latency	1 cycle per stage	5 adder delays, or 5 pipeline stages
Power	Every flop toggles each clock	Every adder switches each clock

2.4 Accuracy of the non-recursive form

Only the final rounding adds error, and it does not accumulate. The SQNR is therefore flat in time and improves with output word length (Figure 3). Each extra bit adds roughly 6 dB, as expected for a quantiser (the console output shows steps of 5.9, 5.8, 5.5 and 4.7 dB from 8 to 12 bits).

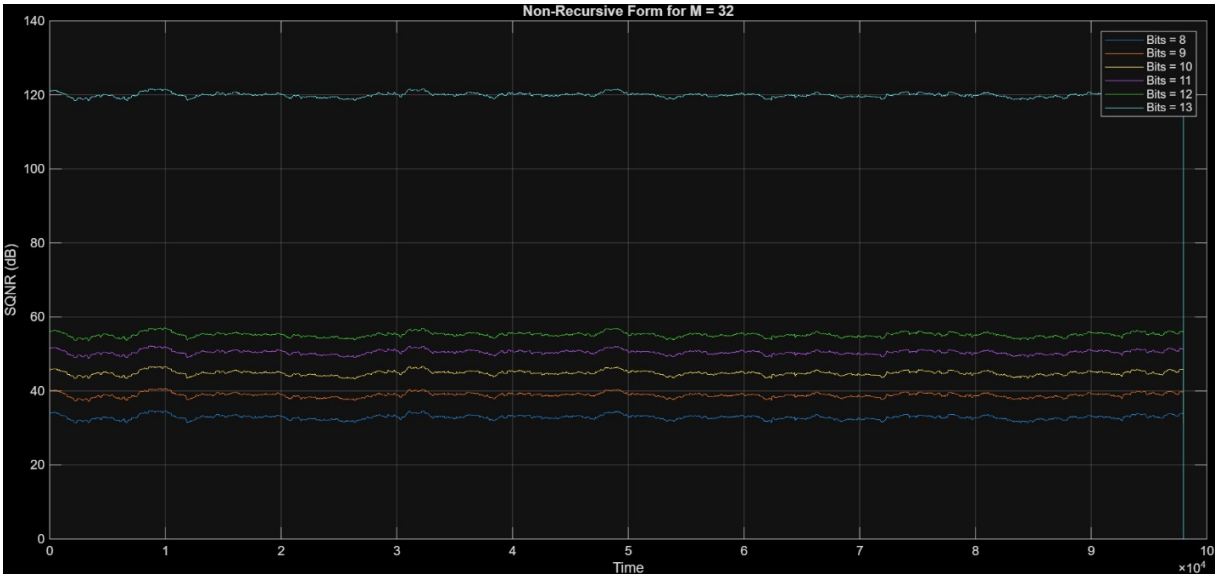


Figure 3. *Non-recursive form, M = 32: SQNR versus time for output word lengths of 8 to 13 bits (MATLAB output).*

At 13 bits the output grid (12 fractional bits) is fine enough to hold the exact average of 32 values that each have 7 fractional bits, so the rounding error is exactly zero. The ≈120 dB curve is not a measured performance. It is the numerical floor of the script, explained in Section 8.

2.5 Drawbacks

- 31 adders and a 31-register parallel-tap delay line: large area, high switching power.
- Long combinational path, or added pipeline latency and registers if the tree is pipelined.
- Cost scales linearly with M, so long windows become impractical.

3. Recursive Form

3.1 Derivation and block diagram

Two successive window sums differ by only two samples: the newest enters and the oldest leaves. Subtracting y[n − 1] from y[n] gives the recursion:

$$y[n] = y[n-1] + (x[n] - x[n-M]) / M$$

Each output needs **one subtraction, one addition and one accumulator register**, independent of M.

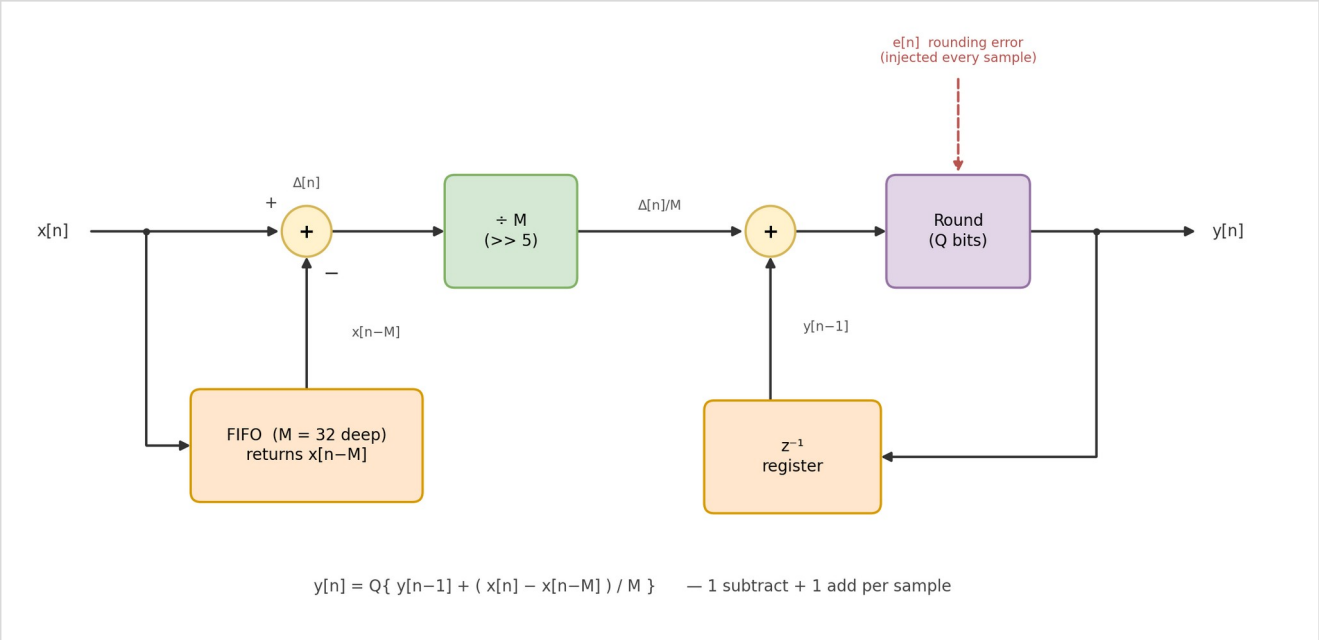


Figure 4. *Recursive moving average. The rounding error e[n] enters inside the feedback loop.*

3.2 What it saves, and what it does not

- **Saves:** the 31-adder tree is replaced by two small adders. The difference needs 9 bits and the accumulator 13 bits.
- **FIFO is still needed:** x[n − M] has to come from somewhere, so M × 8 bits of storage remain. The difference is that only the oldest sample is read each cycle, so a single-read-port circular buffer (register file or RAM) is enough and most of the storage does not toggle every clock.

3.3 Drawback: the SQNR degrades with time

The quantiser sits **inside the feedback loop**. Every sample rounds the new accumulator value, and that rounding error e[n] is never removed: the next sample starts from the rounded value and rounds again. The loop acts as a perfect integrator for its own rounding noise:

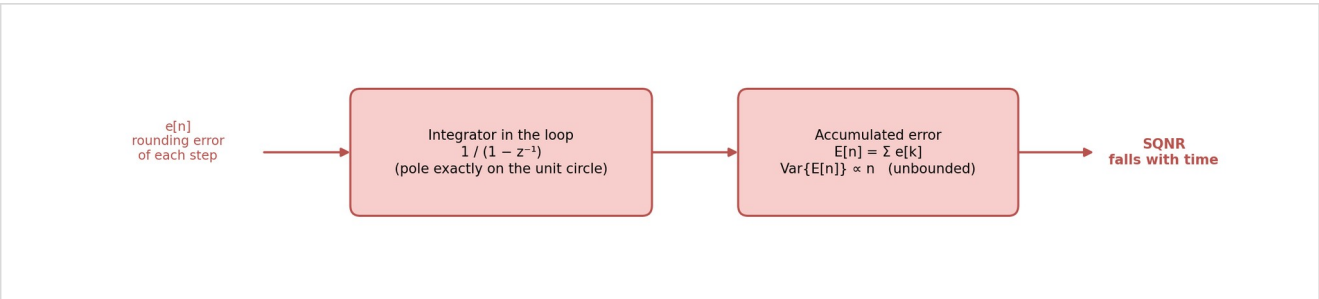


Figure 5. *Error model of the recursive form: the loop integrates its own rounding error.*

$$E[n] = \Sigma\, e[k] \quad (k \le n) \qquad Var\{E[n]\} = n \cdot \sigma^2, \quad \sigma^2 = \Delta^2 / 12 \quad (random\ walk)$$

Even for ideal zero-mean, uncorrelated rounding errors the error power grows **linearly with n** and the SQNR falls by about 3 dB every time the elapsed time doubles. If the rounding errors have a nonzero mean, the error grows linearly and its power grows as n^2, i.e. 6 dB per doubling. The filter has a pole on the unit circle: it never forgets an error.

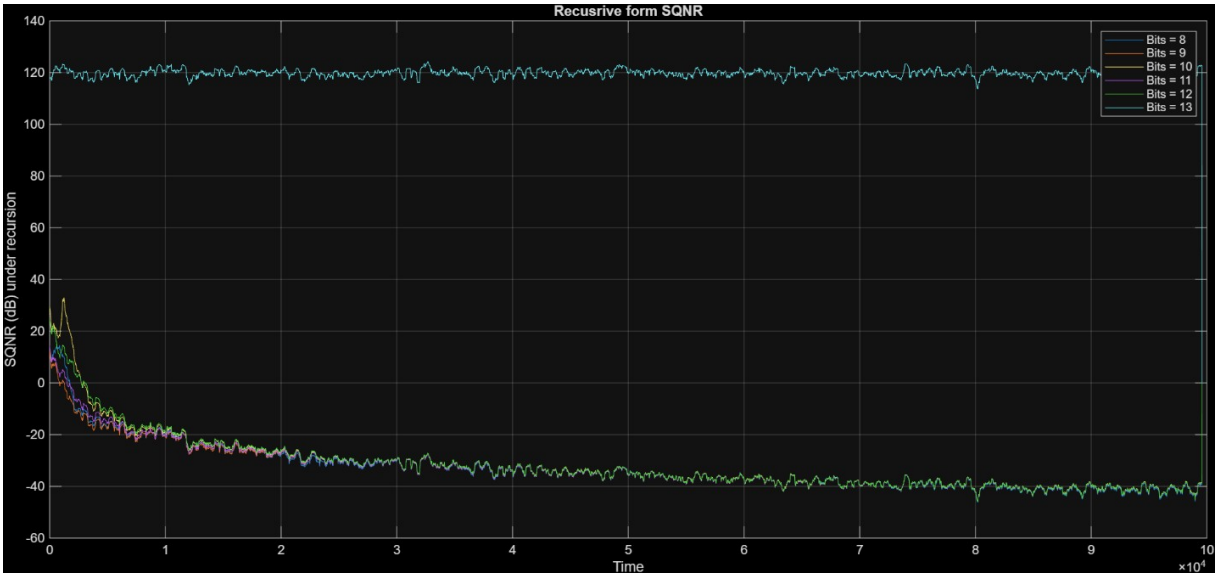


Figure 6. *Recursive form: SQNR versus time. All word lengths up to 12 bits start between roughly 0 and +30 dB and drift below −40 dB; only the 13-bit (exact) case stays flat (MATLAB output).*

Figure 6 shows exactly this. In the first few hundred samples the SQNR is still positive, within about 2,000 samples it drops below 0 dB, and by 10^5 samples it is near −40 dB, which means the accumulated error is far larger than the signal. The averages in the console output are −32.80, −32.87, −31.73, −32.66 and −31.84 dB for 8 to 12 bits. Only 13 bits stays flat (119.34 dB) because its rounding error is identically zero.

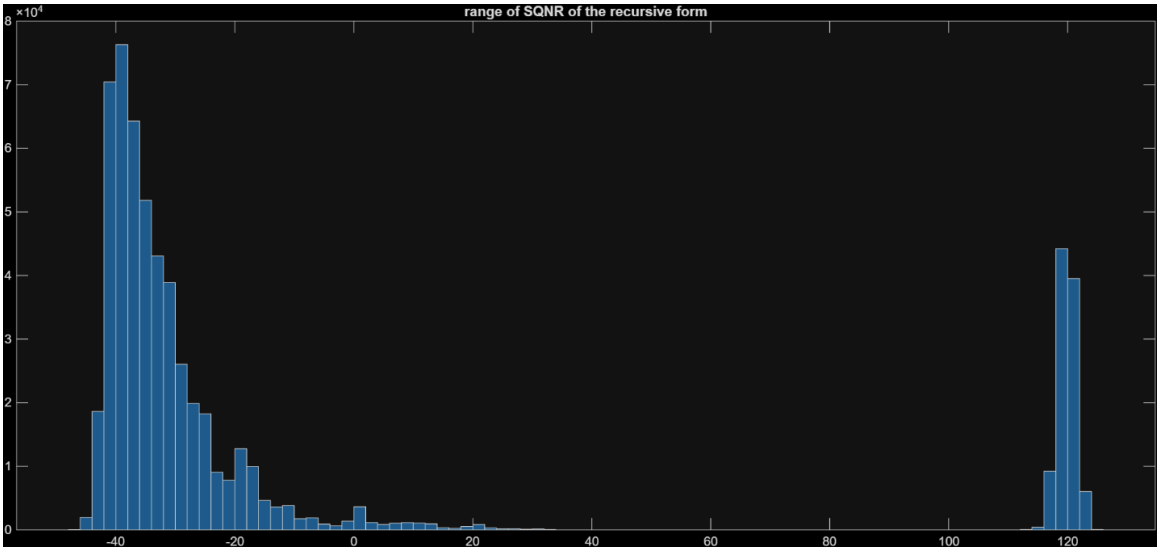


Figure 7. *Histogram of the recursive-form SQNR values over time, all word lengths. Almost all mass lies between −45 and 0 dB; the spike near 120 dB is the exact 13-bit case (MATLAB output).*

Observation: why all word lengths collapse onto one curve

A pure random walk would give 6 dB more SQNR per extra bit, but in Figure 6 the 8 to 12-bit curves lie on top of each other. An independent Python replica of the same loop (not the original MATLAB run) points to

the cause: with 8-bit input, every increment Δ/M lands exactly on a multiple of 2^{-12}, so many values fall **exactly halfway** between two output levels. MATLAB `round()` resolves every tie away from zero, which makes the error correlated with the sign of the data and gives a **systematic drift** that does not shrink with the word length.

In the replica, the error after 10^5 samples was about 12 (normalised units, against a signal RMS near 0.1) for every width from 8 to 12 bits. Replacing the tie rule with round-half-to-even cut it to between 0.01 and 1.5, and random real-valued inputs gave 0.01 to 0.6. An unbiased rounding rule therefore helps, but it does not remove the random-walk growth, which is why the refresh below is still needed.

3.4 Drawbacks

- The error is **unbounded**: SQNR decays with time, and any single upset (a flipped bit, a transient) persists forever.
- It is sensitive to the rounding rule: biased tie-breaking turns a slow random walk into a steady drift.
- The FIFO of M samples is still required, although it can be read at one tap.
- Exactness is only possible if the accumulator is wide enough for zero rounding error (13 bits here), which gives up the word-length saving.

4. Side-by-Side Comparison

Table 3. Comparison of the three architectures.

Criterion (M = 32)	Non-recursive	Recursive	Recursive + refresh
Arithmetic per sample	31 adds	1 subtract + 1 add	1 subtract + 1 add (+ 1 serial add per refresh cycle)
Sample storage	32 × 8 bits, all taps in parallel	32 × 8 bits, one read tap	32 × 8 bits, two read points
Critical path	5 adders (or 5 pipeline stages)	1 add + round	1 add + round
Error behaviour	Bounded, one rounding	Unbounded, grows with n	Bounded, resets every C samples
SQNR at 10 bits	43.96 dB	−31.73 dB	30.79 dB (C = 32)
SQNR at 12 bits	54.12 dB	−31.84 dB	30.53 dB (C = 64)
Verdict	Accurate, area and power heavy	Cheap, unusable on its own	Cheap and meets 30 dB with the right C

5. Solution: Recursive Form with a Counter-Controlled Refresh

5.1 Idea

Keep the cheap recursion, but every C samples **discard the accumulated error** by reloading the output with the exact, freshly rounded full sum:

$$\textit{every } C \textit{ samples:}\quad y[n] \leftarrow Q\{\ (1/M)\cdot\textstyle\sum x[n-k]\ \}$$

$$\textit{otherwise:}\quad y[n] = Q\{\ y[n-1] + (\ x[n] - x[n-M]\ )\ /\ M\ \}$$

Right after a refresh the error is just one rounding (≤ Δ/2). Over the next C samples it grows like a random walk, and then it is reset again. The error is therefore **bounded by the refresh interval** instead of growing with the run length. Figure 8 illustrates the difference on the 10-bit design.

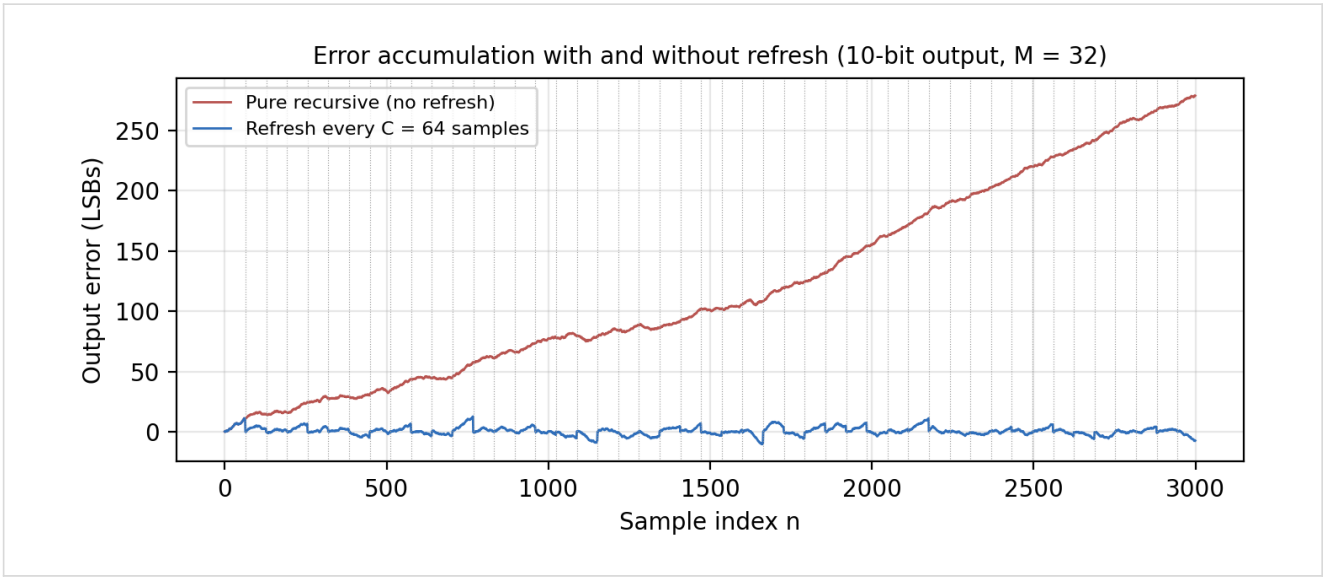


Figure 8. *Output error in LSBs over the first 3,000 samples (10-bit output, M = 32, from a Python replica of the MATLAB loop). Without refresh the error drifts away; with C = 64 it stays bounded. Dotted lines mark refresh instants.*

To a first approximation the error power averaged over one refresh cycle is (C + 1)/2 times the single-rounding power, so every doubling of C costs about 3 dB. The sweep in Section 6 confirms this: between consecutive counter sizes the SQNR drops by about 2.3 to 4.3 dB, close to 3 dB for most steps.

5.2 Architecture

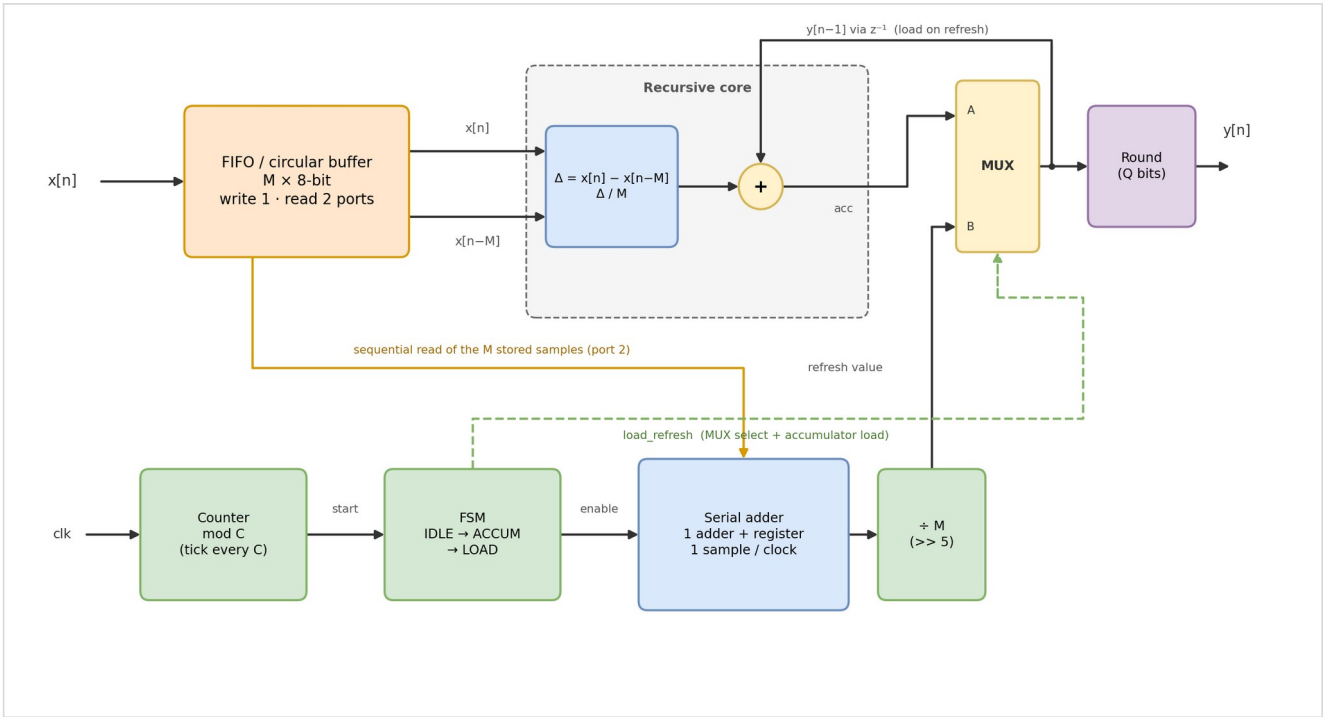


Figure 9. *Recursive core plus refresh path: counter, FSM, serial adder and output MUX.*

The recursive core runs continuously. A **counter** ticks every C clock cycles and starts the refresh FSM. The FSM enables a **serial adder** that reads the stored samples out of the FIFO one per clock and accumulates them. After division by M the exact value is selected by the MUX and loaded into the accumulator.

5.3 Why an open-loop counter

An earlier idea was to monitor the SQNR and refresh only when it drops. That monitor needs power estimates (multipliers, a lot of silicon) plus an error estimator. A **blind timer** replaces the whole monitor with one small integer counter, and the sweep shows that a fixed C already guarantees the target. The counter size is simply a design parameter: larger C means fewer refreshes but lower SQNR.

5.4 The serial adder: full-sum refresh without the tree

The refresh needs the sum of M samples, which is exactly what the 31-adder tree computes. Rebuilding the tree would bring back all the cost the recursion removed. Instead the refresh uses **resource folding**: a single adder, one accumulator register and a tiny FSM add the M samples one after another, one per clock cycle, taking M = 32 cycles.

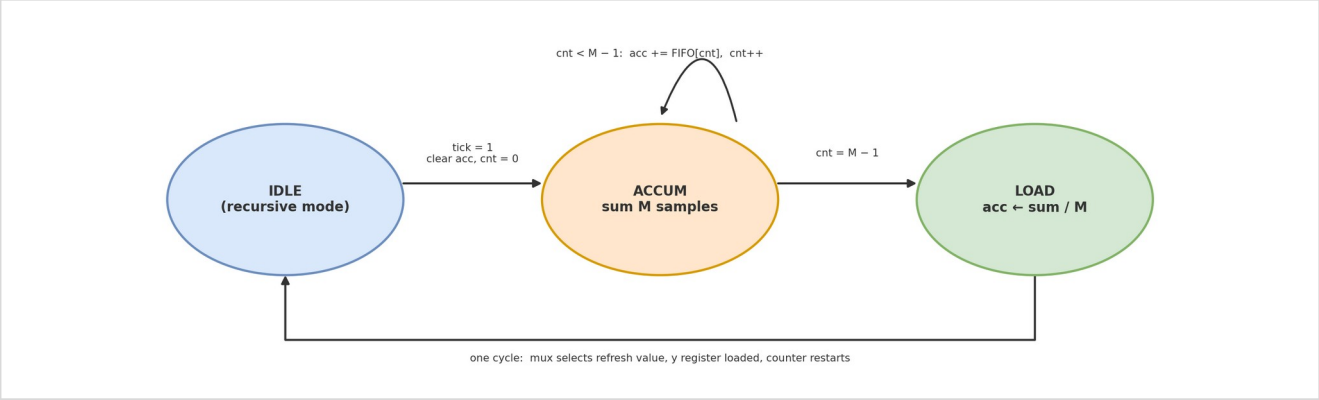


Figure 10. *Refresh FSM: IDLE (recursive mode), ACCUM (serial sum of M samples), LOAD (reload the output).*

- **Area:** one adder, one register and a small FSM instead of 31 adders and, if pipelined, 305 extra flip-flops.
- **No added latency on the output path:** the main output keeps being produced by the recursion while the serial adder works in the background. There is no deep combinational path and no pipeline stages in front of y[n].
- **Cost of folding:** the refresh value is ready M cycles after the tick, so the refresh interval has to satisfy **C ≥ M** if the serial adder runs in the background (Section 7).

> **Implementation note (not modelled in MATLAB)**
>
> The script applies each refresh instantly. In hardware the serial sum covers a window that is M samples old by the time it finishes. Two ways to handle this: (a) stall the input for M cycles during the refresh, which is simple but trades throughput for area, or (b) keep a small shadow accumulator that adds up the increments Δ/M produced during the M cycles and adds it to the serial sum at LOAD, so the stream never stalls. Option (b) is the one that avoids latency.

6. Counter Sweep Results

6.1 Setup

For every counter size C ∈ {2, 4, …, 512} and every output word length (8 to 13 bits) the refresh filter was run over the 100,000 samples, counting refresh events and computing the SQNR over the whole run. Figure 11 shows the instantaneous SQNR at 11 bits for all nine counters. Smaller counters sit higher, and the SQNR is jagged because the 200-sample power window follows the random input power.

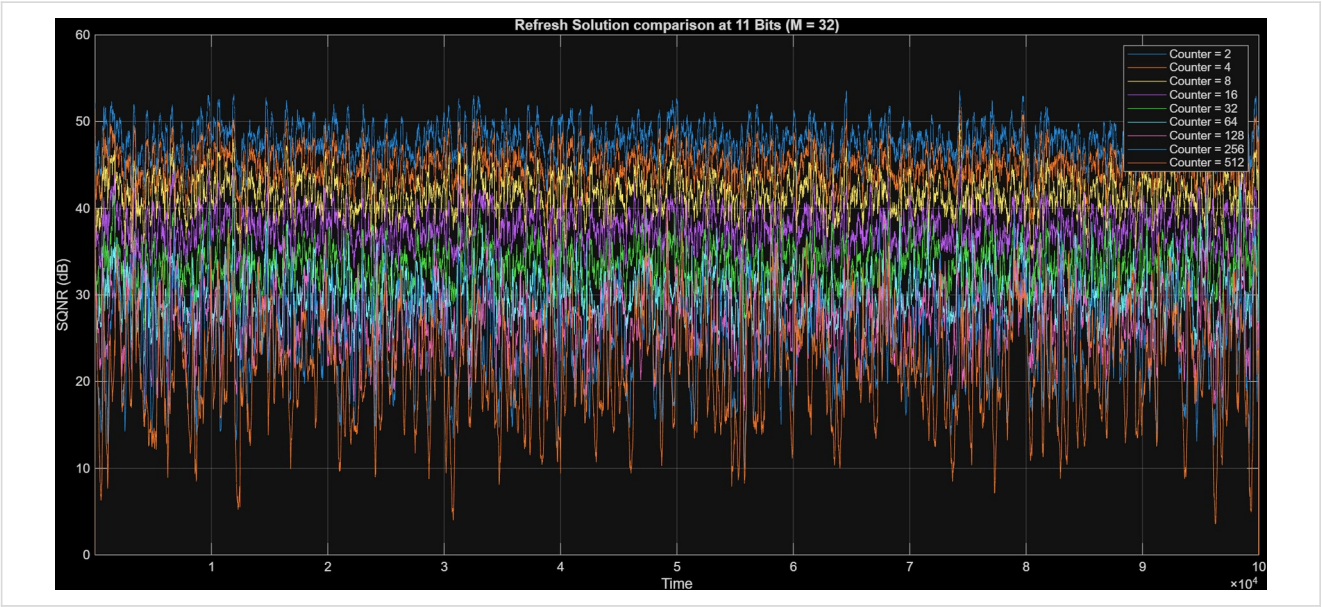


Figure 11. *Refresh solution at 11 bits (M = 32): windowed SQNR versus time for counter sizes 2 to 512 (MATLAB output).*

6.2 SQNR for every counter size and word length

Table 4. Refresh SQNR in dB (target 30 dB). Green cells meet the target, red cells miss it.

Counter C	Refreshes	8 bits	9 bits	10 bits	11 bits	12 bits	13 bits
2	49,984	31.18	37.15	43.05	48.41	52.48	120.04
4	24,992	28.92	34.84	40.63	45.51	48.78	120.04
8	12,496	26.33	32.29	37.76	42.03	44.36	120.04
16	6,248	23.47	29.34	34.42	38.01	39.60	120.04
32	3,124	20.48	26.23	30.79	33.77	34.82	120.04
64	1,562	17.46	22.93	27.26	29.84	30.53	120.04
128	781	14.63	19.70	23.69	26.44	26.87	120.04
256	390	11.07	16.22	20.37	22.77	23.16	120.04
512	195	7.53	12.21	16.59	18.48	19.06	120.04

Figure 12 plots the same data against the refresh interval. The curves fall by roughly 3 dB per doubling of C and cross the dashed 30 dB line at different points for each word length. (The legend entry `data1` is MATLAB's automatic name for the target line.)

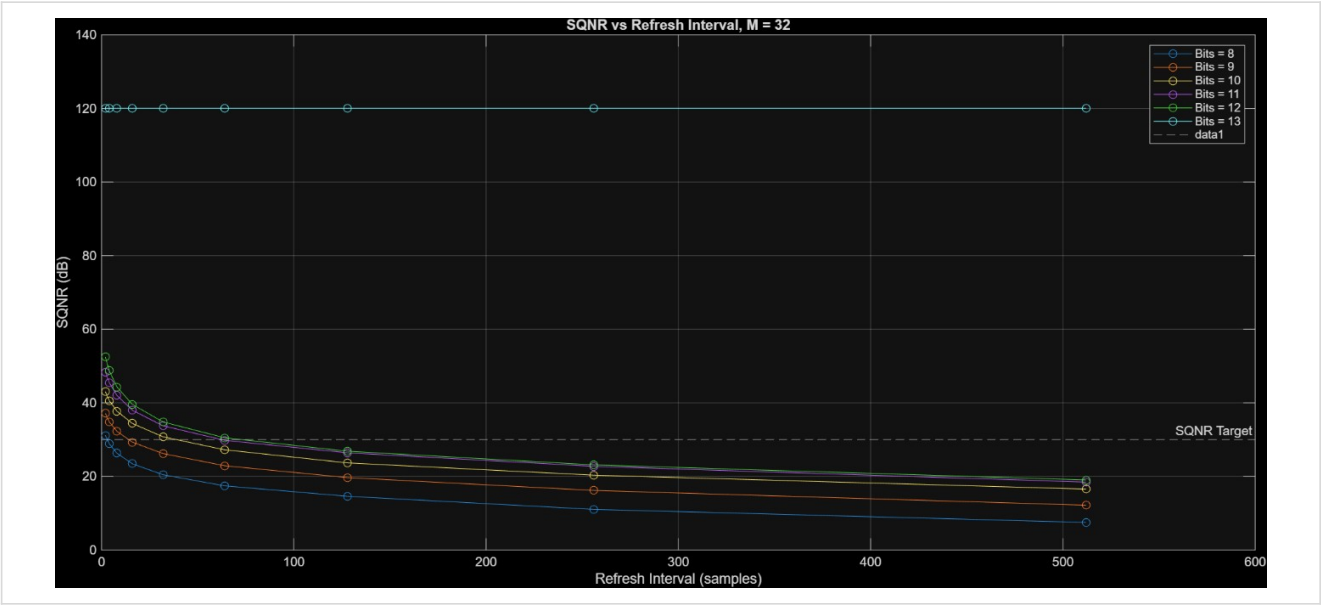


Figure 12. *SQNR versus refresh interval for 8 to 13 bits, with the 30 dB target (MATLAB output).*

6.3 The cases that reach the 30 dB target

Table 5. **Best refresh point per word length ("Best Refresh Point" block of the console output).**

Bits	Largest C meeting 30 dB	SQNR (dB)	Refreshes	Comment
8	2	31.18	49,984	Only C = 2 passes; C = 4 already gives 28.92 dB. A refresh every other sample is as heavy as no recursion at all, and the plain non-recursive form already reaches 32.24 dB.
9	8	32.29	12,496	C = 16 misses at 29.34 dB. One more bit lets the refresh interval grow four times.
10	32	30.79	3,124	Passes with only 0.79 dB margin; C = 64 gives 27.26 dB.
11	32	33.77	3,124	C = 64 misses by 0.16 dB (29.84 dB). The extra bit buys margin (+3 dB) but not a larger power-of-two interval.
12	64	30.53	1,562	0.53 dB margin; C = 128 gives 26.87 dB. The gain over 11 bits is smaller (≈ 4 dB) because the rounding error is no longer uniform.
13	512 (largest tested)	120.04	195	Exact arithmetic: 120.04 dB at every C, so no refresh is needed in this model.

Reading the trends:

- **+6 dB per bit, −3 dB per doubling of C:** one extra bit is worth two doublings of C. That matches 8 → 9 bits (C from 2 to 8) and 9 → 10 bits (8 to 32), but not 10 → 12 bits, where the measured gain is smaller.
- **The 12-bit case:** the increments have 12 fractional bits and are rounded to 11, so half the steps incur zero error and half incur exactly ±Δ/2. This gives an error variance of $\Delta^2/8$ instead of $\Delta^2/12$ and costs about 1.8 dB, which is why the 11 → 12-bit gain is only about 4 dB.
- **The 13-bit case:** the SQNR is the script's numerical floor, not a measurement (Section 8).

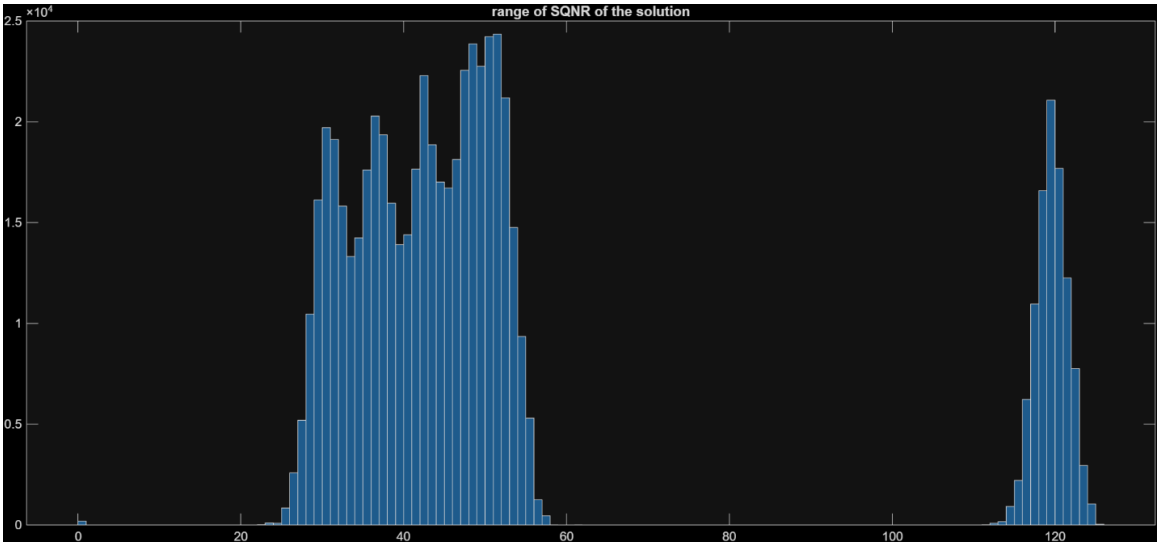


Figure 13. *Histogram of the refresh-solution SQNR (counter size 2, all word lengths): the 25 to 58 dB cluster spans the 8 to 12-bit designs, and the spike near 120 dB is the exact 13-bit case (MATLAB output).*

Compare Figure 13 with Figure 7: the recursive form spread over −45 to +30 dB, while the refreshed form stays within 25 to 58 dB, entirely above the target for the best word lengths. The tiny bar at 0 dB comes from unfilled array entries at the start and end of the run.

7. Serial-Adder Feasibility and Practical Choice

A serial adder adds one sample per clock, so a refresh takes M = 32 cycles. If it runs in the background, the refresh period must be at least M. Dividing M by C gives the duty cycle of the serial adder:

Table 6. Can a single serial adder serve the best counter size?

Bits	Best C	Duty M / C	Background?	Comment
8	2	1600 %	No	Needs a parallel sum; defeats the purpose
9	8	400 %	No	Same problem; the serial adder cannot keep up
10	32	100 %	Just	Serial adder is always busy; no slack and 0.79 dB margin
11	32	100 %	Just	No slack, 3.77 dB margin
12	64	50 %	Yes	Half the time idle; 0.53 dB margin
13	512	6 %	Yes	Exact, refresh not required

With a single serial adder, the practical designs are therefore **10 or 11 bits with C = 32** (serial adder fully busy) or **12 bits with C = 64** (serial adder half idle). Refreshing every 64 samples instead of every sample cuts the refresh rate to 1/64, replaces 31 adders by one, and still meets 30 dB. If more SQNR margin is wanted, 12 bits with C = 32 gives 34.82 dB at the cost of keeping the serial adder 100 % busy.

8. Measurement Notes and Caveats

1. **The ≈120 dB values are a floor, not a result.** The script clamps the error power at 10^{-14} before taking the logarithm. With zero error, the SQNR is $10 \cdot \log_{10}(P_y / 10^{-14}) \approx 120$ dB for $P_y \approx 0.0104$, regardless of architecture or C.

2. **Baseline averages are slightly low.** The windowed SQNR array is only filled up to index N − M + 1 − 1999, but the average is taken over 99,969 entries, so about 2,000 zero entries (2 %) are included. That is the vertical drop at the right edge of Figure 3, and it is why the 13-bit non-recursive average reads 117.58 dB while the plot sits near 120 dB.
3. **Forward-looking window.** The non-recursive loop averages x[i … i+M−1], which equals the causal filter delayed by M − 1 samples. SQNR is unaffected.
4. **Rounding rule.** `round()` ties away from zero. Hardware often truncates or rounds half up. All of these are biased, and the biased drift described in Section 3.3 may differ, but the refresh bounds it in every case.
5. **Instantaneous refresh.** The model refreshes in one step. The serial-adder timing (Section 5.4) is not simulated.
6. **Single random run.** The script does not average over many input sequences. Margins below about 1 dB (10 bits / C = 32, 12 bits / C = 64) should be re-checked with several seeds before being trusted.
7. **Average versus worst case.** The target is checked on the whole-run average. Figure 11 shows the instantaneous SQNR swinging by several dB around that average, so a worst-case specification would call for a smaller C.

9. Conclusion

- The **non-recursive** form is accurate (SQNR flat, +6 dB per bit) but needs 31 adders, a 5-level tree and a 31-register delay line that toggles every clock.
- The **recursive** form needs only one add and one subtract per sample, but its feedback integrates the rounding error. SQNR degrades continuously and averages about −32 dB for 8 to 12 bits.
- A **counter-controlled refresh** every C samples bounds the error. The sweep shows the 30 dB target is reached at 8 bits (C = 2), 9 bits (C ≤ 8), 10 and 11 bits (C ≤ 32), and 12 bits (C ≤ 64); each doubling of C costs about 3 dB and each extra bit gains about 6 dB.
- Computing the refresh sum with a **serial adder** and an FSM replaces the adder tree with one adder, and running it in the background keeps the output path free of added latency. This requires C ≥ M, which makes 10 to 12 bits with C = 32 to 64 the practical operating region.

Appendix A. Complete MATLAB Console Output

Output of the script for M = 32, reproduced verbatim.

```
================== FINAL RESULTS FOR M = 32 ==================

--- Baseline Forms ---
Bits: 8 | Non-Rec SQNR: 32.24 dB | Pure Rec SQNR: -32.80 dB
Bits: 9 | Non-Rec SQNR: 38.13 dB | Pure Rec SQNR: -32.87 dB
Bits: 10 | Non-Rec SQNR: 43.96 dB | Pure Rec SQNR: -31.73 dB
Bits: 11 | Non-Rec SQNR: 49.46 dB | Pure Rec SQNR: -32.66 dB
Bits: 12 | Non-Rec SQNR: 54.12 dB | Pure Rec SQNR: -31.84 dB
Bits: 13 | Non-Rec SQNR: 117.58 dB | Pure Rec SQNR: 119.34 dB

--- Refresh Solutions (Sweep Counter Size) ---

Counter Wait Size: 2 Samples
  Bits:  8 | Refresh SQNR: 31.18 dB | Refreshes Occurred: 49984
  Bits:  9 | Refresh SQNR: 37.15 dB | Refreshes Occurred: 49984
  Bits: 10 | Refresh SQNR: 43.05 dB | Refreshes Occurred: 49984
  Bits: 11 | Refresh SQNR: 48.41 dB | Refreshes Occurred: 49984
  Bits: 12 | Refresh SQNR: 52.48 dB | Refreshes Occurred: 49984
  Bits: 13 | Refresh SQNR: 120.04 dB | Refreshes Occurred: 49984

Counter Wait Size: 4 Samples
  Bits:  8 | Refresh SQNR: 28.92 dB | Refreshes Occurred: 24992
  Bits:  9 | Refresh SQNR: 34.84 dB | Refreshes Occurred: 24992
  Bits: 10 | Refresh SQNR: 40.63 dB | Refreshes Occurred: 24992
  Bits: 11 | Refresh SQNR: 45.51 dB | Refreshes Occurred: 24992
  Bits: 12 | Refresh SQNR: 48.78 dB | Refreshes Occurred: 24992
  Bits: 13 | Refresh SQNR: 120.04 dB | Refreshes Occurred: 24992

Counter Wait Size: 8 Samples
  Bits:  8 | Refresh SQNR: 26.33 dB | Refreshes Occurred: 12496
  Bits:  9 | Refresh SQNR: 32.29 dB | Refreshes Occurred: 12496
  Bits: 10 | Refresh SQNR: 37.76 dB | Refreshes Occurred: 12496
  Bits: 11 | Refresh SQNR: 42.03 dB | Refreshes Occurred: 12496
  Bits: 12 | Refresh SQNR: 44.36 dB | Refreshes Occurred: 12496
  Bits: 13 | Refresh SQNR: 120.04 dB | Refreshes Occurred: 12496

Counter Wait Size: 16 Samples
  Bits:  8 | Refresh SQNR: 23.47 dB | Refreshes Occurred: 6248
  Bits:  9 | Refresh SQNR: 29.34 dB | Refreshes Occurred: 6248
  Bits: 10 | Refresh SQNR: 34.42 dB | Refreshes Occurred: 6248
  Bits: 11 | Refresh SQNR: 38.01 dB | Refreshes Occurred: 6248
  Bits: 12 | Refresh SQNR: 39.60 dB | Refreshes Occurred: 6248
  Bits: 13 | Refresh SQNR: 120.04 dB | Refreshes Occurred: 6248

Counter Wait Size: 32 Samples
  Bits:  8 | Refresh SQNR: 20.48 dB | Refreshes Occurred: 3124
  Bits:  9 | Refresh SQNR: 26.23 dB | Refreshes Occurred: 3124
  Bits: 10 | Refresh SQNR: 30.79 dB | Refreshes Occurred: 3124
  Bits: 11 | Refresh SQNR: 33.77 dB | Refreshes Occurred: 3124
  Bits: 12 | Refresh SQNR: 34.82 dB | Refreshes Occurred: 3124
  Bits: 13 | Refresh SQNR: 120.04 dB | Refreshes Occurred: 3124

Counter Wait Size: 64 Samples
  Bits:  8 | Refresh SQNR: 17.46 dB | Refreshes Occurred: 1562
  Bits:  9 | Refresh SQNR: 22.93 dB | Refreshes Occurred: 1562
  Bits: 10 | Refresh SQNR: 27.26 dB | Refreshes Occurred: 1562
  Bits: 11 | Refresh SQNR: 29.84 dB | Refreshes Occurred: 1562
  Bits: 12 | Refresh SQNR: 30.53 dB | Refreshes Occurred: 1562
  Bits: 13 | Refresh SQNR: 120.04 dB | Refreshes Occurred: 1562

Counter Wait Size: 128 Samples
```

```
  Bits:  8 | Refresh SQNR: 14.63 dB | Refreshes Occurred: 781
  Bits:  9 | Refresh SQNR: 19.70 dB | Refreshes Occurred: 781
  Bits: 10 | Refresh SQNR: 23.69 dB | Refreshes Occurred: 781
  Bits: 11 | Refresh SQNR: 26.44 dB | Refreshes Occurred: 781
  Bits: 12 | Refresh SQNR: 26.87 dB | Refreshes Occurred: 781
  Bits: 13 | Refresh SQNR: 120.04 dB | Refreshes Occurred: 781

Counter Wait Size: 256 Samples
  Bits:  8 | Refresh SQNR: 11.07 dB | Refreshes Occurred: 390
  Bits:  9 | Refresh SQNR: 16.22 dB | Refreshes Occurred: 390
  Bits: 10 | Refresh SQNR: 20.37 dB | Refreshes Occurred: 390
  Bits: 11 | Refresh SQNR: 22.77 dB | Refreshes Occurred: 390
  Bits: 12 | Refresh SQNR: 23.16 dB | Refreshes Occurred: 390
  Bits: 13 | Refresh SQNR: 120.04 dB | Refreshes Occurred: 390

Counter Wait Size: 512 Samples
  Bits:  8 | Refresh SQNR:  7.53 dB | Refreshes Occurred: 195
  Bits:  9 | Refresh SQNR: 12.21 dB | Refreshes Occurred: 195
  Bits: 10 | Refresh SQNR: 16.59 dB | Refreshes Occurred: 195
  Bits: 11 | Refresh SQNR: 18.48 dB | Refreshes Occurred: 195
  Bits: 12 | Refresh SQNR: 19.06 dB | Refreshes Occurred: 195
  Bits: 13 | Refresh SQNR: 120.04 dB | Refreshes Occurred: 195
============================================================

--- Best Refresh Point ---
Bits:  8 | Best Counter:    2 | SQNR: 31.18 dB | Refreshes: 49984
Bits:  9 | Best Counter:    8 | SQNR: 32.29 dB | Refreshes: 12496
Bits: 10 | Best Counter:   32 | SQNR: 30.79 dB | Refreshes: 3124
Bits: 11 | Best Counter:   32 | SQNR: 33.77 dB | Refreshes: 3124
Bits: 12 | Best Counter:   64 | SQNR: 30.53 dB | Refreshes: 1562
Bits: 13 | Best Counter:  512 | SQNR: 120.04 dB | Refreshes: 195
```